Author Support & Communication Portal: Approach Note

Anju Kumari · Full Stack Developer assignment ·

- Live demo: https://author-support-portal-delta.vercel.app/·
- Code: https://github.com/logicscienc/author-support-portal

What I built

A ticketing portal where authors raise queries about their books and BookLeaf's team resolves them faster. Authors see their books, royalties and tickets. Admins get a prioritised queue in which every ticket arrives classified, scored and with a drafted reply based on BookLeaf's policies and the author's real data. Stack: React, Node/Express, Prisma, PostgreSQL (Neon) and OpenAI gpt-4o-mini.

What I prioritised

- **The core flow first:** login, raise a ticket, AI triage, admin reply, and the author seeing it within 10 seconds.
- **AI quality, the highest-weighted criterion.** Facts a model should not calculate are calculated in code and handed over as facts: the payout deadline, the Rs 1,000 minimum check and how long ago a book was published. Early drafts got these wrong, and testing each fix on repeated drafts of the same tickets removed the errors. Each call sends only the relevant policy section (about 440 tokens to classify and 1,180 to draft), and drafts are saved so reopening a ticket costs nothing.
- **Resilience:** timeout, retry and a circuit breaker, with keyword rules as a fallback. A ticket is always created, and if drafting fails the admin writes the reply by hand.
- **Security:** role checks and ownership filters in the queries, so authors cannot see other authors' tickets. Internal notes and AI fields never reach authors, and the API key stays on the server.
- **Dataset edge cases:** books in production, zero payouts, and pending royalty above or below the minimum.

Trade-offs

- Polling every 10 seconds instead of WebSockets: simpler and reliable on free hosting, and close enough to real time.
- Classification runs in the web process, not a job queue: quicker to build. If the server restarts mid-call, the ticket shows "AI pending" until an admin opens it.
- Drafts are always human-reviewed. I still saw small slips in testing, so the UI warns the admin and records whether a reply came from a draft.
- No automated tests due to time. I tested manually, including the failure cases.
- Attachments are UI only, as the brief allows.

Taking it to production

- Move AI calls to a queue with retries. Build an evaluation set of real tickets to track classification accuracy, and add an automatic check that compares a draft's figures and dates with the data before it is shown.
- Email authors when they get a reply, and use server-sent events instead of polling.
- Real file uploads, an audit trail (the table already exists), httpOnly-cookie sessions, Redis-backed rate limits, API tests with CI, and monitoring of AI cost and failure rate.